Veritas: Ontological Reasoning for Evidence-Backed Mathematical Research-to-Code Systems

Research-to-Production Gap for Autonomous Agent Architecture

Veritas Contributors
Open-source repository: https://github.com/daddydrac/veritas

June 2026

Abstract

Enterprises increasingly use generative and agentic artificial intelligence (AI) to accelerate software engineering, but recent industry reports converge on a common constraint: productivity tools do not reliably become organizational value unless development workflows are redesigned around evidence, governance, process integration, and accountable execution. This paper presents VERITAS, an open-source research-to-code architecture that combines Dockerized services, vLLM model serving, OpenSearch FAISS/HNSW vector retrieval, Apache Jena Fuseki RDF/SPARQL graphs, Openllet ontology reasoning, Docling-first PDF processing, formula-aware chunking, and a bounded planning-code-validation loop. The central contribution is an ontological reasoning model that connects objectives, evidence, symbolic formulas, invariants, risks, plans, task specifications, source code artifacts, validation checks, and build artifacts. We formalize this model using typed graph and obligation semantics, describe the implementation state of the supplied repository snapshot, present clean architecture diagrams and command-line states, and argue that VERITAS is superior to prompt-to-code assistants for math-heavy internal algorithm development because it reasons over obligations rather than merely over retrieved text. The paper is intentionally academic rather than promotional: it identifies implemented capabilities, host-validation requirements, limitations, and open research problems in ontological mathematical reasoning for software engineering. **Keywords:** ontology engineering, mathematical software engineering, agentic AI, vLLM, OpenSearch, Jena Fuseki, RDF, SPARQL, research-to-code, invariant preservation, evidence-backed code generation.

1 Introduction

Every technical company eventually accumulates an algorithm graveyard. A scientist writes a promising method in a notebook, an engineer turns part of it into a prototype, and a team embeds the prototype inside an internal tool. Months later, the organization cannot confidently answer which paper the method came from, which formula was implemented, which assumptions were preserved, which invariants were tested, which risks were accepted, or whether the implementation can scale. This is not merely a documentation failure. It is a semantic-continuity failure.

Recent enterprise research motivates this problem from several directions. Bain & Company reports that many software organizations have rolled out generative AI tools, but developer adoption remains uneven and productivity gains often stay modest unless the development process itself changes; assistant-based gains are frequently around 10–15%, while larger improvements are associated with deeper process redesign (Bain & Company, 2025a). Bain separately characterizes agentic AI as a structural shift that requires modern foundations, interoperability, security, and accountability before enterprises can safely deploy agents that reason, coordinate, and execute complex workflows (Bain & Company, 2025b). Boston Consulting Group argues that only a small share of organizations are truly generating AI value at scale, while many lag because they have not redesigned operating models, data systems, and technology foundations (Boston Consulting Group, 2025). McKinsey frames AI's software-development opportunity as transformation of the full product development life cycle, not isolated code completion (Gnanasambandam et al., 2025). Google's DORA research emphasizes that AI's impact on software development is nuanced across individuals, teams, and organizations (Google Research, n.d.), and Reuters reports Gartner's projection that many agentic AI projects may be scrapped because of unclear value, cost, immature capabilities, and "agent washing" (Singh, 2025).

VERITAS is motivated by that gap. It is designed for mathematical research and proprietary algorithm development, where ordinary coding assistance is insufficient. These domains require a system that preserves the reasoning chain from research objective to evidence, formula, invariant, risk, plan, generated source code, validation checks, and build artifact. The guiding thesis is:

> Research should not become software by losing meaning. Research should become software by preserving meaning.

The repository snapshot evaluated for this paper is the supplied `veritas-autonomous-e2e-no-placeholders` codebase. It contains a Rust API and CLI, Docker Compose services, Python ingestion workers, an ontology package, SPARQL queries, OpenSearch vector indexing logic, vLLM model-serving services, and tests for ingestion and code-generation utilities. The snapshot implements a source-level end-to-end path in which `/run` creates a workspace, requests a structured plan, validates plan JSON, executes planner-selected tools, writes model-generated files, runs compile/test commands, feeds fail-

ures back to the code model, retries a bounded number of times, and emits a final report. Live certification still requires running Docker, Cargo, GPU resources, and vLLM model loading on a host environment; those facts should be treated as implementation constraints rather than papered over.

Contributions. This paper makes five contributions. First, it defines a formal ontology-guided reasoning model for research-to-code systems. Second, it presents the updated VERITAS architecture and removes unsupported property-graph and non-vLLM model-serving claims. Third, it formalizes the symbolic mathematics represented by the ontology. Fourth, it walks through the CLI states used to make the workflow operational for non-specialists. Fifth, it evaluates the repository snapshot in an academic manner, separating implemented source-level behavior from host-validated production execution.

2 Enterprise Problem Synthesis

The cited studies do not describe VERITAS directly. They describe an enterprise gap that VERITAS targets: the distance between AI experimentation and reproducible, governed, scalable software outcomes. Table 1 summarizes this relationship.

The problem is therefore not that organizations lack models. It is that they lack durable reasoning infrastructure for turning research into production systems. A conventional prompt-to-code workflow can be written as

$$\mathcal{M}(p, D) \to C_{src}, \tag{1}$$

where p is a prompt, D is retrieved context, and C_{src} is generated source code. Equation 1 is underconstrained. It can produce plausible code while dropping assumptions, changing mathematical meaning, ignoring risks, and omitting validation obligations.

For math-heavy research-to-code work, VERITAS seeks the stronger mapping

$$O \to E \to S \to I \to R \to P \to T \to C_{src} \to V \to B, \tag{2}$$

where O is an objective, E is evidence, S is a symbolic shadow, I is an invariant, R is risk, P is a plan, T is a task specification, C_{src} is source code, V is a validation check, and B is a build artifact. Equation 2 expresses the central VERITAS pipeline: a deployable artifact should be downstream of evidence-backed intent, symbolic structure, invariant preservation, risk analysis, executable planning, concrete tasks, generated source, and explicit validation.

3 System Architecture

Figure 1 summarizes the reference architecture. The implementation uses Jena/Fuseki RDF/SPARQL as the implemented graph layer; it does not require a separate property graph service. OpenSearch provides vector and lexical retrieval. vLLM provides model-serving endpoints for planner, code, and math roles. The Rust API owns orchestration, configuration, failure reporting, workspace creation, command execution, bounded repair, and final reporting.

3.1 Implementation key

The supplied repository snapshot implements the following major components:

- **Rust CLI**: guides initialization, service startup, ingestion, model inspection, planning, runs, and status reporting.
- **Rust API**: exposes `/health`, `/models`, `/plan`, `/run`, `/status`, search, SPARQL, and model-chat endpoints.
- **vLLM services**: separate Docker services for planner, code, and math roles, using OpenAI-compatible HTTP interfaces as documented by vLLM (vLLM Project, n.d.a,n).
- **OpenSearch**: default vector memory using FAISS/HNSW approximate nearest-neighbor indexing; OpenSearch documents approximate k-NN search and engine/method configuration for vector retrieval (OpenSearch Project, n.d.a,n).
- **Embedding service**: uses the configured SBERT model for normalized embeddings; the default model is `Muennighoff/SBERT-base-nli-v2` (Hugging Face, n.d.).
- **Jena/Fuseki**: stores RDF triples and exposes SPARQL query/update and graph-store protocols; Fuseki is documented as a SPARQL server by Apache Jena (Apache Software Foundation, n.d.).
- **Docling-first ingestion**: converts PDFs into structured representations and is intended for downstream AI/RAG systems; Docling documents formula, table, reading-order, chunk, and bounding-box support (Docling Project, n.d.).
- **Run workspace**: stores model outputs, generated files, command logs, validation results, retry history, and `final_report.json`.

4 Formal Ontological Model

4.1 Ontology as typed graph and constraint system

Let the VERITAS ontology be a typed graph with axioms:

$$\mathcal{O} = (\mathcal{C}, \mathcal{P}, \mathcal{A}, \mathcal{R}), \tag{3}$$

where $\mathcal{C}$ is a set of classes, $\mathcal{P}$ is a set of object and datatype properties, $\mathcal{A}$ is a set of OWL-DL axioms, and $\mathcal{R}$ is a set of operational constraints enforced by orchestration and validation. The core class vocabulary is

$$\begin{aligned} \mathcal{C}_{core} = \{&\mathsf{Objective}, \mathsf{Plan}, \mathsf{TaskSpecification}, \mathsf{Risk}, \\ &\mathsf{Invariant}, \mathsf{EvidenceArtifact}, \mathsf{ValidationCheckSpecification}, \\ &\mathsf{SymbolicShadow}, \mathsf{SourceCodeArtifact}, \mathsf{BuildArtifact}\}. \end{aligned} \tag{4}$$

The goal is not only classification. The ontology creates obligations. A source-code artifact that implements a formula-derived method should be linked to evidence and validation. A risk that threatens an objective should be mitigated, accepted, or reported as a blocker. An invariant should be tied to a transformation family before the system treats it as mathematically grounded.

Table 1: Enterprise problem statements from recent industry research and the corresponding VERITAS design response.

Source	Problem described	Veritas response
Bain, software development (Bain & Company, 2025a)	Generative AI tools can produce modest developer productivity gains, but adoption and value remain uneven unless the software-development process itself changes.	VERITAS moves beyond assistant-style code completion by linking research evidence, objectives, plans, risks, validation checks, and code artifacts in one reasoning system.
Bain, agentic AI (Bain & Company, 2025b)	Enterprise agents require platforms, interoperability, governance, security, and accountability before safe agentic execution is realistic.	VERITAS uses Dockerized services, vLLM model routing, OpenSearch retrieval, Fuseki ontology graphs, bounded execution, and structured reports to make agent behavior inspectable.
BCG (Boston Consulting Group, 2025)	A small group of companies creates value from AI at scale, while many remain stuck in experiments because foundations and operating models are weak.	VERITAS turns math-heavy R&D into a repeatable workflow: ingest evidence, ground reasoning in an ontology, generate code, validate it, and produce artifacts.
McKinsey (Gnanasambandam et al., 2025)	AI's strongest software-development value comes from full life-cycle transformation, not isolated coding tasks.	VERITAS spans objective definition, evidence retrieval, formula extraction, invariant analysis, planning, code generation, testing, and build-artifact production.
Google DORA (Google Research, n.d.)	AI's impact varies across individuals, teams, and organizations; acceleration without governance can be incomplete or misleading.	VERITAS makes AI output auditable: claims, formulas, plans, code artifacts, and validation results are tied back to evidence and ontology-grounded reasoning.
Reuters/Gartner (Singh, 2025)	Many agentic AI projects may fail due to cost, unclear value, immaturity, and agent washing.	VERITAS treats autonomy as an execution protocol: model routes, tool calls, validations, error states, and retries are explicit rather than hidden behind a slogan.

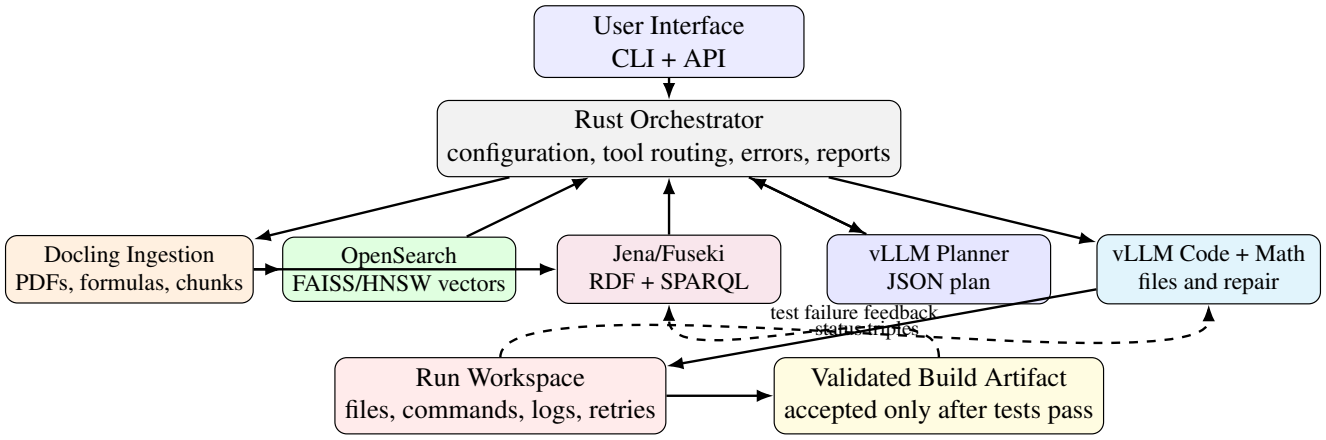

Figure 1: Clean reference architecture for VERITAS. The architecture separates document ingestion, vector retrieval, RDF/SPARQL ontology grounding, vLLM model roles, tool execution, run workspaces, and build-artifact validation.

Let $G = (V, E)$ be the RDF knowledge graph. Each vertex $v \in V$ has a type assignment

$$\tau(v) \in \mathcal{C}, \tag{5}$$

and each edge is a property assertion

$$(v_i, p, v_j) \in E, \quad p \in \mathcal{P}. \tag{6}$$

A planning state is valid only when required ontology paths exist. For example, for a candidate code artifact c implementing symbolic structure s, a minimal obligation can be written as

$$\exists e, I, v \; [\; \mathsf{supports}(e, s) \wedge \mathsf{preserves}(c, I) \wedge \mathsf{validatedBy}(c, v) \;]. \tag{7}$$

Equation 7 is not a proof of correctness. It is a structural requirement that prevents the agent from treating ungrounded code as an accepted build artifact.

4.2 Documents, formulas, and evidence

Given a document d, ingestion produces chunks c_i, formulas σ_j, and evidence artifacts e_k:

$$\Phi(d) = \left(\{c_i\}_{i=1}^{n}, \{\sigma_j\}_{j=1}^{m}, \{e_k\}_{k=1}^{r}\right). \tag{8}$$

A formula is modeled as a symbolic shadow:

$$\sigma \in \mathsf{SymbolicShadow}, \qquad \mathsf{SymbolicShadow} \not\equiv \mathsf{Invariant}. \tag{9}$$

This distinction is central. A formula is evidence of a structure, not the entire structure. The agent must infer or hypothesize relations such as

$$\mathsf{supports}(e, \sigma), \qquad \mathsf{suggests}(\sigma, I), \qquad I \in \mathsf{Invariant}. \tag{10}$$

4.3 Representation maps and invariants

The mathematical-research model is representation-first. Let X_s be a surface representation and X_ℓ be a latent representation. A representation map is

$$R : X_s \to X_\ell. \tag{11}$$

The map is useful when it makes a governing law simpler while preserving the relevant invariant. For an admissible transformation $T \in \mathcal{T}_{adm}$, an invariant is a quantity I satisfying

$$I(T(x)) = I(x), \qquad \forall x \in X,\ T \in \mathcal{T}_{adm}. \tag{12}$$

The code-generation obligation is stronger than syntactic similarity. If g is generated code and f is the intended mathematical operation, validation should establish

$$\Delta\big(I(g(x)), I(f(x))\big) \le \epsilon, \tag{13}$$

for a domain-relevant discrepancy measure Δ and tolerance ϵ.

4.4 Evidence-weighted retrieval

For a query q and chunk c_i, VERITAS combines vector retrieval, lexical retrieval, and graph grounding:

$$S(q, c_i) = \alpha \cos(\mathbf{z}_q, \mathbf{z}_i) + \beta\, \mathrm{BM25}(q, c_i) + \gamma\, G(q, c_i), \tag{14}$$

where

$$\mathbf{z}_i = \frac{f_\theta(c_i)}{\|f_\theta(c_i)\|_2}. \tag{15}$$

The normalization in Equation 15 makes cosine scoring stable for the sentence embedding model. The graph support term $G(q, c_i)$ can encode whether a chunk participates in an RDF path from a source document to a symbolic shadow, invariant, risk, plan, or validation check. Thus retrieval is not only similarity over text; it is similarity plus semantic role.

4.5 Risk, validation, and acceptance

A risk can be scored by likelihood, impact, and uncertainty:

$$\rho(r) = p(r)\, \iota(r)\, u(r), \tag{16}$$

where $p(r)$ is likelihood, $\iota(r)$ is impact, and $u(r)$ is uncertainty or lack of evidence. VERITAS should reduce $\rho(r)$ using mitigation, evidence, tests, or acceptance decisions.

A build artifact is accepted only when validation checks pass:

$$\mathrm{Accept}(B) = \bigwedge_{k=1}^{K} v_k(B) = 1. \tag{17}$$

In the evaluated repository snapshot, generated packages are not supposed to receive a production-candidate status until compile and test commands pass. This is the operational difference between code generation and validated artifact generation.

5 Autonomous Planning and Code Generation

The `/run` endpoint is intended to implement a bounded loop rather than a deterministic envelope. At the source level, the supplied API performs the following sequence:

1. Validate that a user goal is present.
2. Create a unique run workspace under the configured runs directory.
3. Retrieve evidence from OpenSearch before planning.
4. Run a formula-traceability SPARQL query against Fuseki.
5. Call the planner model through vLLM.
6. Validate that the planner response is JSON and satisfies the plan schema.
7. Execute planner-selected tool categories: retrieval, SPARQL, math reasoning, code generation, local command, and test runner.
8. Call the code-generation model and require structured JSON with files and commands.
9. Write files into the run workspace using safe relative paths.
10. Run compile or test commands.
11. Feed failures back to the code model and retry up to a bounded maximum.
12. Write `final_report.json` containing task, plan, model routes, tool calls, files changed, commands run, validation results, retries, final status, and limitations.

Formally, let s_t be the run-workspace state after attempt t, π_t be the planner or repair instruction, and o_t be command output. The autonomous repair loop is

$$s_{t+1} = F(s_t, \pi_t, o_t), \qquad 0 \le t \le K, \tag{18}$$

where K is the maximum retry bound. The loop terminates when

$$\mathrm{Accept}(B_t) = 1 \quad \vee \quad t = K. \tag{19}$$

This avoids unbounded agent execution and creates a failure report when validation cannot be satisfied.

6 CLI State Machine

The CLI is intended to make the workflow reproducible for users who do not want to manage OpenSearch mappings, Fuseki graph URLs, vLLM endpoints, embedding caches, or formula chunking manually. Table 2 summarizes all states at a glance using the supplied CLI walkthrough as the design source, but converts it to an academic table rather than a marketing graphic.

7 Repository Snapshot Evaluation

7.1 Source-level implementation state

The supplied codebase includes the following implemented service definitions in `docker-compose.yml`: `opensearch`, `dashboards`, `fuseki`, `embedding`,

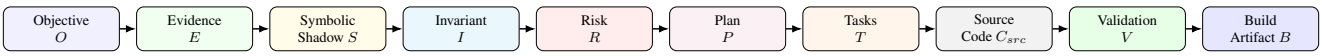

Figure 2: Ontology-guided reasoning pipeline. The chain expresses the obligations that must be represented before research-derived code is accepted as a build artifact.

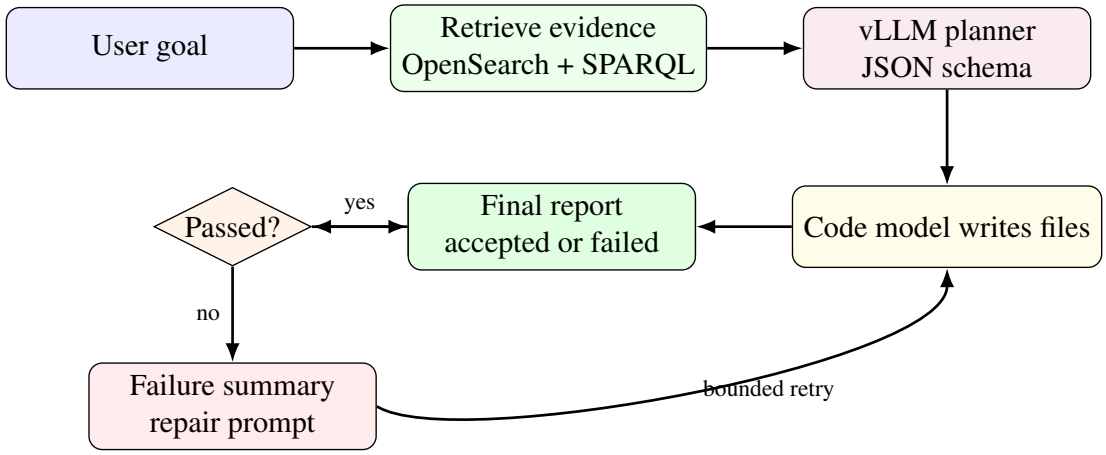

Figure 3: Bounded autonomous planning and coding loop. Failures are fed back to the code model, but the loop is bounded and always emits a report.

`api`, `cli`, `ingestion`, `reasoner`, `vllm-planner`, `vllm-code`, and `vllm-math`. The vLLM services are partitioned by role and expose planner, code, and math model endpoints. The default planner model is `Qwen/Qwen2.5-Coder-7B-Instruct`; the default code model is `Qwen/Qwen2.5-Coder-14B-Instruct`; the default math model is `allenai/Olmo-3-7B-Instruct`; and the default embedding model is `Muennighoff/SBERT-base-nli-v2`.

The Rust API source exposes endpoints for health, readiness, model configuration, graph status, retrieval, SPARQL, model chat, planning, and autonomous runs. The inspected `/run` path creates a workspace, invokes planning, executes tools, writes final reports, and records files and commands. The CLI source includes a banner, an initialization workflow, model configuration prompts, functional-programming design questions, and environment persistence. The generated configuration includes settings for a functional-core / imperative-shell architecture, referential transparency where practical, dispatch tables, higher-order functions, and isolated side-effect boundaries.

7.2 Validation boundary

A careful academic evaluation must distinguish three levels:

1. **Design claim**: the architecture specifies a behavior.
2. **Source-level implementation**: code exists to execute the behavior.
3. **Host-validated production execution**: Docker, model weights, GPU runtime, Cargo, OpenSearch, Fuseki, vLLM, and ingestion all run successfully on a target host.

The supplied environment for this paper did not provide Docker, Cargo, or GPU runtime. Therefore, claims here are based on repository inspection and the available prior test reports, not on a live host execution. Production sign-off requires commands such as `docker compose up`, `cargo check`, vLLM model loading, OpenSearch indexing, Fuseki SPARQL querying, PDF ingestion, and an actual `/run` workflow to pass on the target host.

7.3 Implemented and unresolved concerns

The supplied snapshot addresses several previously identified placeholder gaps at the source level: `/run` creates a workspace, planning is model-backed through vLLM, generated code is written into run workspaces, compile/test commands are executed, failures are fed back into bounded retries, and final reports include files and commands. At the same time, formula extraction remains best-effort, model quality depends on selected weights and prompts, and live production proof depends on host validation. The paper therefore does not claim that VERITAS solves theorem proving or arbitrary production software generation. It claims that VERITAS implements a stronger research-to-code reasoning architecture than prompt-to-code workflows because it records and enforces semantic obligations.

8 Why Veritas Is Superior for Its Target Use Cases

8.1 Compared with prompt-to-code

Prompt-to-code systems optimize for immediacy. VERITAS optimizes for traceability and correctness obligations. If an implementation fails later, VERITAS can help isolate whether the fault came from formula extraction, missing assumptions, wrong representation, numerical instability, code generation, dependency mismatch, or inadequate validation.

Table 2: CLI states at a glance. The state machine makes configuration, ingestion, planning, execution, retry, and audit inspection visible to the user.

No.	State	System action	Business or research outcome
1	Welcome banner	Prints VERITAS identity, version, and help prompt.	Confirms that the platform is installed and ready.
2	Init start	Starts interactive configuration.	Avoids hidden defaults and records user intent.
3	Role models	Selects planner, code, and math models.	Controls cost, quality, latency, and role specialization.
4	Embeddings and paths	Selects embedding model, token, and cache path.	Makes retrieval reproducible and model downloads governed.
5	Services	Configures OpenSearch, Fuseki, and supporting endpoints.	Centralizes retrieval, graph reasoning, and service control.
6	Document ingestion	Configures PDF ingestion, chunking, overlap, and formula extraction.	Converts research artifacts into searchable evidence.
7	Ontology ingestion	Loads OWL/RDF/TTL into Fuseki.	Makes domain reasoning explicit.
8	Init summary	Reviews settings before persistence.	Reduces misconfiguration.
9	Init complete	Saves configuration and suggests next commands.	Makes setup repeatable.
10	Non-interactive init	Supports one-command configuration.	Enables CI, scripted demos, and reproducible deployments.
11	Start services	Launches Docker Compose services.	Makes the platform available.
12	Health check	Checks vLLM, OpenSearch, Fuseki, embedding, ingestion, and reasoner services.	Surfaces failures early with remediation.
13	List models	Shows model roles, IDs, providers, and endpoints.	Gives operators model visibility.
14	Plan	Planner returns structured JSON.	Provides an auditable plan before code is written.
15	Run	Executes planner-selected steps.	Tracks work end-to-end.
16	Fail and retry	Feeds compile/test failures back to code model.	Repairs errors instead of hiding them.
17	Final report	Produces files changed, commands, validations, retries, and final status.	Provides reproducible audit evidence.
18	Runs list	Displays historical runs.	Enables tracking outcomes and regressions.
19	Run status	Shows detailed run workspace and logs.	Supports debugging and compliance review.
20	Config and commands	Provides configuration, query, validation, logs, and service controls.	Lets users operate the system without editing source code.

8.2 Compared with ordinary RAG

Ordinary RAG retrieves chunks and passes them to a model. VERITAS adds typed semantic commitments. A chunk can become an evidence artifact; a formula can become a symbolic shadow; a symbolic shadow can suggest an invariant; an invariant can constrain code; and a validation check can determine whether a build artifact is accepted. That is not merely more context. It is a typed obligation graph.

8.3 Compared with agentic demos

Agentic demos often hide their execution model. VERITAS exposes it. The planner returns structured JSON, the run loop creates a workspace, tools execute with logs, commands run in the workspace, failures are fed back to the model, package status changes only after validation succeeds, and the final report records the process.

8.4 Compared with ungoverned research prototypes

Research prototypes optimize for local demonstration. VERITAS optimizes for reproducibility, auditability, and scaling. The ontology requires the agent to reason about objectives, risks, invariants, evidence, and validation before treating code as a build artifact.

9 Limitations and Open Research Problems

VERITAS does not solve mathematical truth by itself. It can extract symbolic shadows, retrieve evidence, identify candidate invariants, ask configured models for reasoning, execute validation commands, and record results. Theorem-level claims still require proof, formal verification, counterexample search, or human review.

Formula extraction is a hard problem. Docling-style extraction is useful for converting documents into structured data and extracting mathematical formulas, but PDF mathematics is not guaranteed to be recovered perfectly across all corpora (Docling Project, n.d.). Therefore VERITAS treats formula extraction as best-effort and requires golden tests for target corpora.

Generated code quality depends on evidence quality, model quality, validation commands, and test strictness. A weak test suite can still accept a weak implementation. Future work should therefore add property-based tests, numerical validation generators, formal specification integrations, proof-assistant links, and domain-specific correctness benchmarks.

GPU execution claims must remain constrained. The current architecture wires vLLM GPU model serving and exposes tensor-parallel and GPU-memory knobs. It should not claim that arbitrary generated application code has a working CUDA, Candle, Burn, or wgpu backend unless such code is generated and validated in a workspace.

Open research contributions are needed in at least eight areas: richer OWL-DL modeling of transformations and invariants; stronger formula, diagram, and source-TeX extraction; formal links between symbolic shadows and executable code; benchmarks for research-paper-to-code conversion; property-based and numerical test generation; proof assistant integration; model-routing policies; and enterprise governance for security, observability, and traceability.

10 Conclusion

The gap between research and production is not only a software-engineering gap. It is a semantic-continuity gap. Organizations lose value when mathematical assumptions, formulas, evidence, risks, and validation obligations fail to survive the transition into code. VERITAS addresses this by combining ontology-guided

reasoning, evidence-backed retrieval, model-routed planning, and bounded code-validation loops.

The key improvement is that VERITAS reasons about obligations, not just content. It asks what must be true before research becomes production-grade software. It then makes those requirements queryable, testable, auditable, and connected to deployable artifacts. That is why ontology matters, and that is why research-to-code systems need more than ordinary RAG or ordinary code generation.

References

Bain & Company. (2025a, September 23). *From pilots to payoff: Generative AI in software development*. Bain & Company. https://www.bain.com/insights/from-pilots-to-payoff-generative-ai-in-software-development-technology-report-2025/

Bain & Company. (2025b, September 23). *Building the foundation for agentic AI*. Bain & Company. https://www.bain.com/insights/building-the-foundation-for-agentic-ai-technology-report-2025/

Boston Consulting Group. (2025, September 30). *Are you generating value from AI? The widening gap*. Boston Consulting Group. https://www.bcg.com/publications/2025/are-you-generating-value-from-ai-the-widening-gap

Docling Project. (n.d.). *Docling*. https://www.docling.ai/

Gnanasambandam, C., Harrysson, M., Singh, R., & Chawla, A. (2025, February 10). *How an AI-enabled software product development life cycle will fuel innovation*. McKinsey & Company. https://www.mckinsey.com/industries/technology-media-and-telecommunications/our-insights/how-an-ai-enabled-software-product-development-life-cycle-will-fuel-innovation

Google Research. (n.d.). *DORA impact of generative AI in software development*. Google Research. https://research.google/pubs/dora-impact-of-generative-ai-in-software-development/

Apache Software Foundation. (n.d.). *Apache Jena Fuseki*. https://jena.apache.org/documentation/fuseki2/

OpenSearch Project. (n.d.a). *Approximate k-NN search*. https://docs.opensearch.org/latest/vector-search/vector-search-techniques/approximate-knn/

OpenSearch Project. (n.d.b). *Methods and engines*. https://docs.opensearch.org/latest/mappings/supported-field-types/knn-methods-engines/

Singh, J. (2025, June 25). *Over 40% of agentic AI projects will be scrapped by 2027, Gartner says*. Reuters. https://www.reuters.com/business/over-40-agentic-ai-projects-will-be-scrapped-by-2027-gartner-says-2025-06-25/

Veritas Project. (2026). *Veritas: Ontological reasoning for mathematical research to production code* [Computer software]. GitHub. https://github.com/daddydrac/veritas

vLLM Project. (n.d.a). *Using Docker*. https://docs.vllm.ai/en/stable/deployment/docker/

vLLM Project. (n.d.b). *OpenAI-compatible server*. https://docs.vllm.ai/en/stable/serving/openai_compatible_server/

Hugging Face. (n.d.). *Muennighoff/SBERT-base-nli-v2*. https://huggingface.co/Muennighoff/SBERT-base-nli-v2